

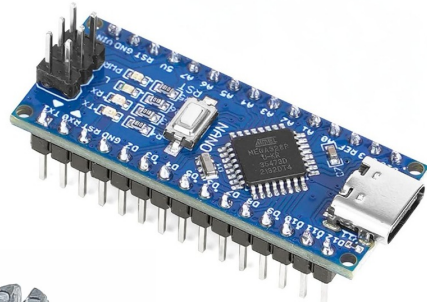
Lab 4: Analog input

Lab goals

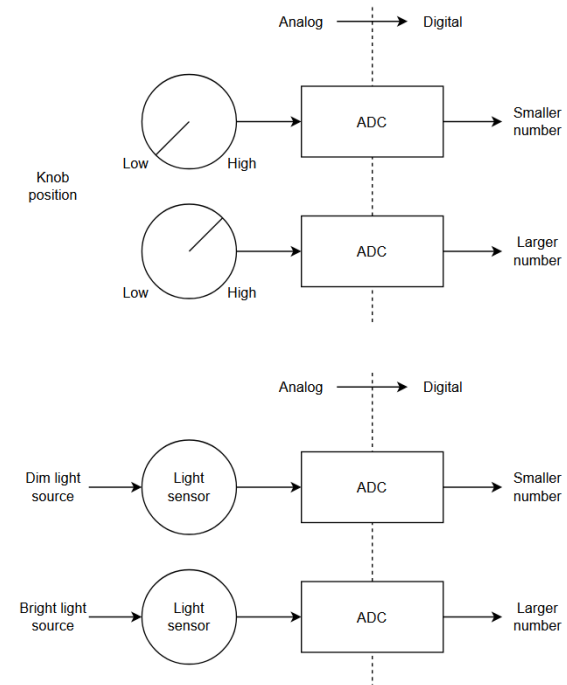
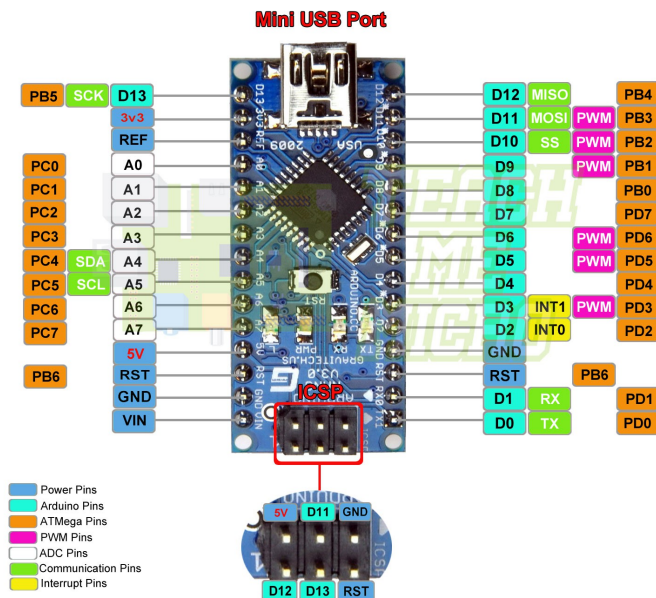
- Get acquainted with the microcontroller's capability for reading and measuring analog input
- Get acquainted with USB Serial and the Arduino IDE's "Serial Monitor"

Required hardware/software

- Everything from lab 2, plus:
- Potentiometer (variable resistor) x 1



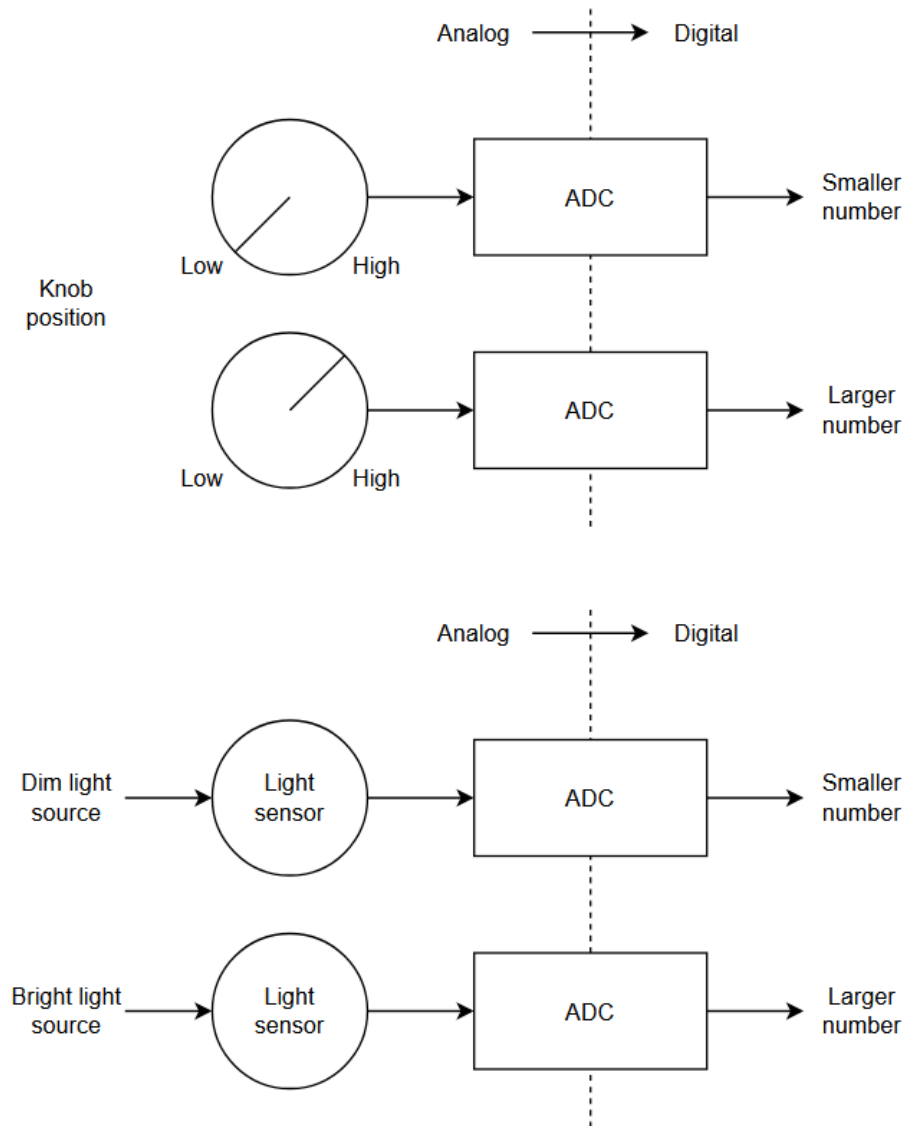
ARDUINO NANO PINOUT



Lab 4: Analog input (background)

Recall that most microcontrollers have a built-in analog-to-digital converter (ADC) for reading an analog voltage level and converting it into a digital value (integer).

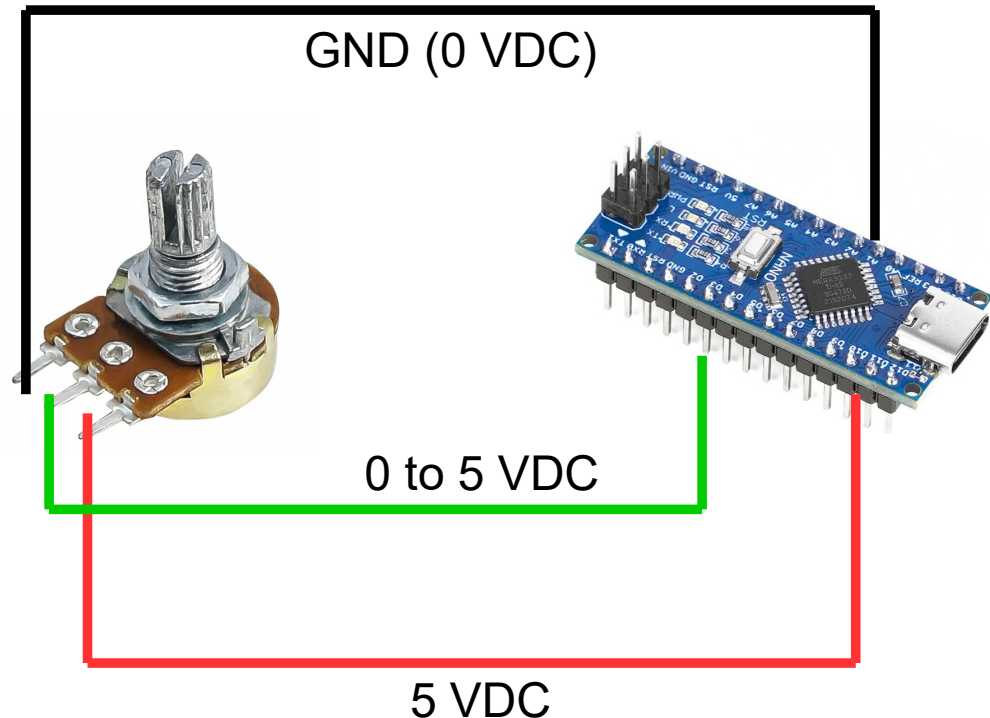
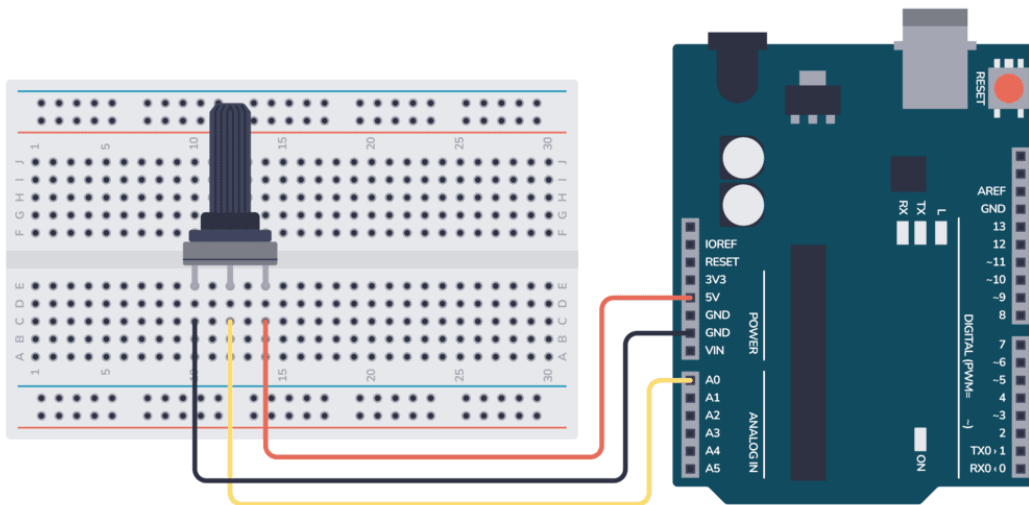
Computers (microcontrollers) don't deal in truly analog quantities (continuous) but instead use digital (discrete) approximations of them.



Lab 4: Analog input (background)

At one end of how far the knob will turn, the ADC will be presented with 0 VDC. At the other end of how far the knob will turn, the ADC will be presented with 5 VDC.

Remember: The knob (potentiometer) is simply a stand-in for any number of different possible analog voltage sources (light sensors, temperature reading resistors, etc).



Lab 4: Analog input (background)

Also recall that the Arduino Nano development board is based on the Microchip (Atmel) Atmega328P microcontroller. The Atmega328P has a built-in 10 bit analog-to-digital converter (ADC).

More on the meaning of “10 bit” ADC on the next slide.

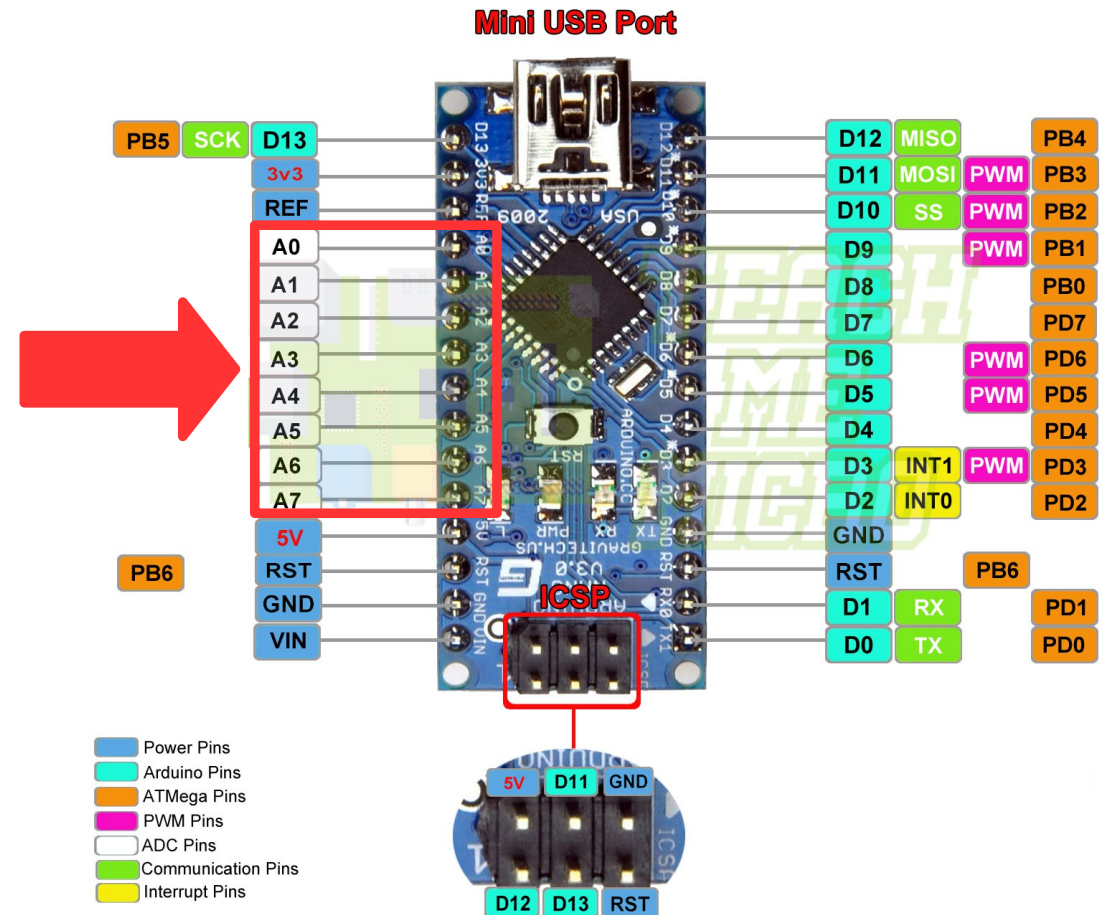
The pins marked “A0” through “A7” are directly connected to the Atmega328P's built-in ADC.

Each one of these pins represents a different ADC “channel.”

The Atmega328P's ADC has 8 individual channels, meaning that 8 different analog sources can be read and converted to digital values.

A0 represents channel 1, A1 = channel 2, A2 = channel 3, ..., A7 = channel 8.

ARDUINO NANO PINOUT



Lab 4: Analog input (background)

The more “bits” an ADC has, the “finer” a reading it can perform of an analog voltage. We refer to this as the “resolution” of the ADC. The Atmega328P's built-in ADC has 10 bits of resolution.

The table below shows how many different values can be represented by a given number of bits, from 1 bit up to 12 bits (although higher bit resolutions are not uncommon e.g. 16 and 24 bit ADCs).

<u># bits</u>	<u># different possible values</u>
1	2
2	4
3	8
4	16
5	32
6	64
7	128
8	256
9	512
10	1,024
11	2,048
12	4,096
...	
24	16,777,216

This generalizes to the following equation:

different possible values = 2 to the n^{th} power,
where n = number of bits

For example:

6 bits: 2 to the 6^{th} power = $2 \times 2 \times 2 \times 2 \times 2 \times 2$,
which equals 64

Lab 4: Analog input (background)

A 10 bit number can be used to represent any integer between 0 and 1,023.

A 10 bit ADC will read 0 at the lowest voltage it can measure (0 VDC) and will read 1,023 at the highest voltage it can measure (in this case, 5 VDC).

Likewise, It will read:

- One quarter of 1,023 (~ 256) when measuring one quarter of 5 VDC (1.25 VDC)
- Half of 1,023 (~ 512) when measuring half of 5 VDC (2.5 VDC)
- Three quarters of 1,023 (~ 768) when measuring three quarters of 5 VDC (3.75 VDC)
- And so on and so forth

If we divide 5 VDC by 1,024 then we get approximately 5 mV, which is the approximately the smallest possible change in voltage that a 10 bit ADC can measure over a 5 volt range.

Q: What would a 2 bit ADC do?

If we considered an 8 bit ADC (256 possible values) then we would see that the smallest possible change in voltage that it can measure over a 5 volt range would be approximately 20 mV, or approximately 4X less resolution than the 10 bit ADC.

In general, the more bits of ADC resolution the better, but only up until a certain point.

Lab 4: Analog input (background)

USB Serial

Many development boards allow for a “USB Serial” connection, even for microcontrollers that don't natively support USB. In the case of microcontrollers without native USB support, it is typical for a “USB to UART bridge” IC to be present, but the details of this are out of scope for this introductory course. Common USB to UART bridge ICs include Silicon Lab's CP210x series, FTDI's FT232 series, and the CH340 series.

USB Serial is used mostly to exchange text between a microcontroller and a PC, although binary information can also be exchanged.

One useful application of being able to exchange text between a microcontroller and a PC is for the purposes of debugging, which is especially useful when developing microcontroller firmware without access to a debugger (which, for example, the Arduino IDE historically lacked) and/or without access to some other form of input/output to/from the microcontroller.

Common ways to access the USB Serial connection to a microcontroller include the “PuTTY” application for Windows and the Arduino IDE's built-in “Serial Monitor.”

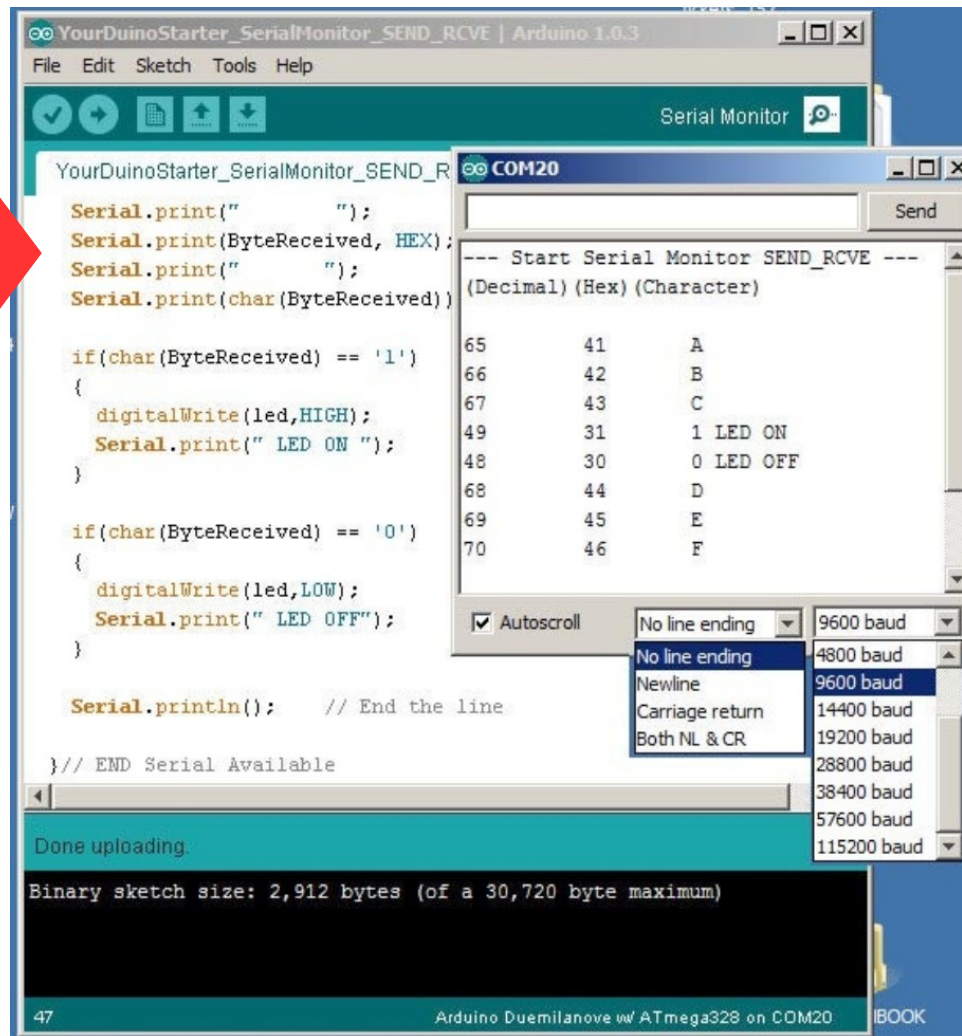
(Next slide)

Lab 4: Analog input (background)

USB Serial (continued)

The screenshot below shows the Serial Monitor open in Arduino IDE. It also shows the **Serial.print()** and **Serial.println()** statements added to the code, which allow one to easily include this sort of output from their microcontroller. More on this a bit later.

Serial.print() and
Serial.println()
statements in code



Arduino IDE's
Serial Monitor

Lab 4: Analog input (background)

USB Serial

Many development boards allow for a “USB Serial” connection, even for microcontrollers that don't natively support USB. In the case of microcontrollers without native USB support, it is typical for a “USB to UART bridge” IC to be present, but the details of this are out of scope for this introductory course. Common USB to UART bridge ICs include Silicon Lab's CP210x series, FTDI's FT232 series, and the CH340 series.

USB Serial is used mostly to exchange text between a microcontroller and a PC, although binary information can also be exchanged.

One useful application of being able to exchange text between a microcontroller and a PC is for the purposes of debugging, which is especially useful when developing microcontroller firmware without access to a debugger (which, for example, the Arduino IDE historically lacked) and/or without access to some other form of input/output to/from the microcontroller.

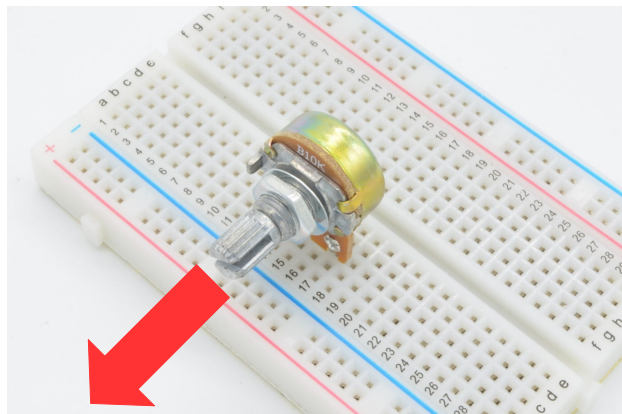
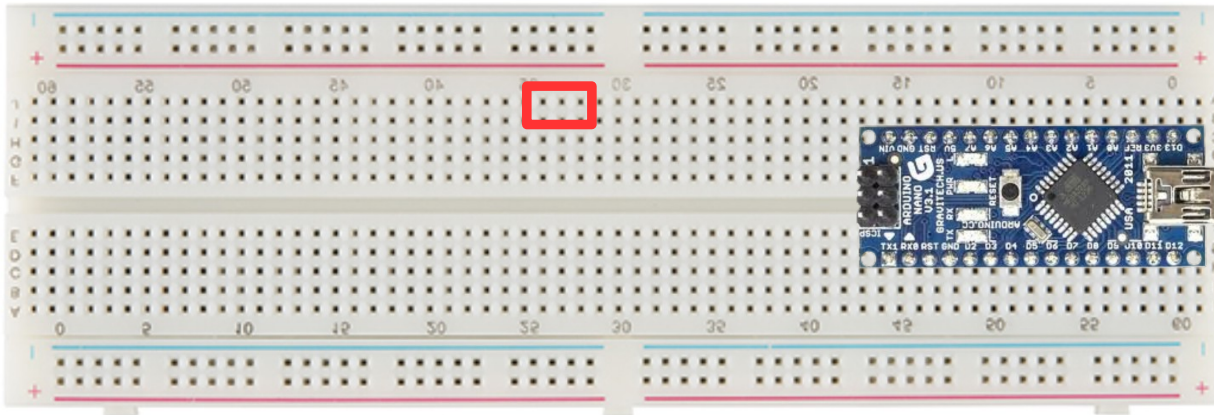
Common ways to access the USB Serial connection to a microcontroller include the “PuTTY” application for Windows and the Arduino IDE's built-in “Serial Monitor.”

(Next slide)

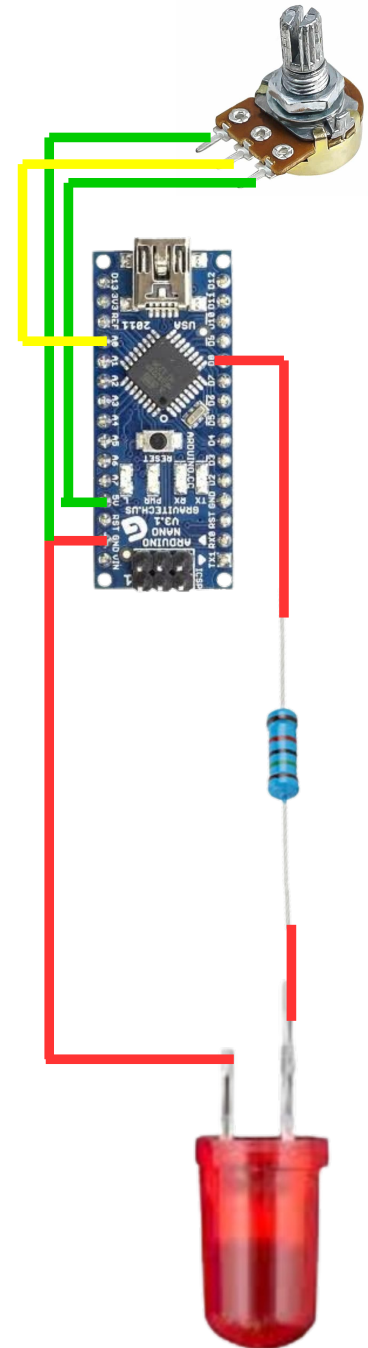
Lab 4: Analog input

Steps

1. As a starting point, disconnect the USB cable from your development board and leave the components and connections from lab 2 intact. Remove lab 3's button and associated wires.
2. Choose three rows on your breadboard and insert your potentiometer



Choose an outer-most column so that the knob of the potentiometer hangs over the side of your breadboard (unlike what is shown in the image to the left) and so that you also maximize the number of holes you can use behind the potentiometer.



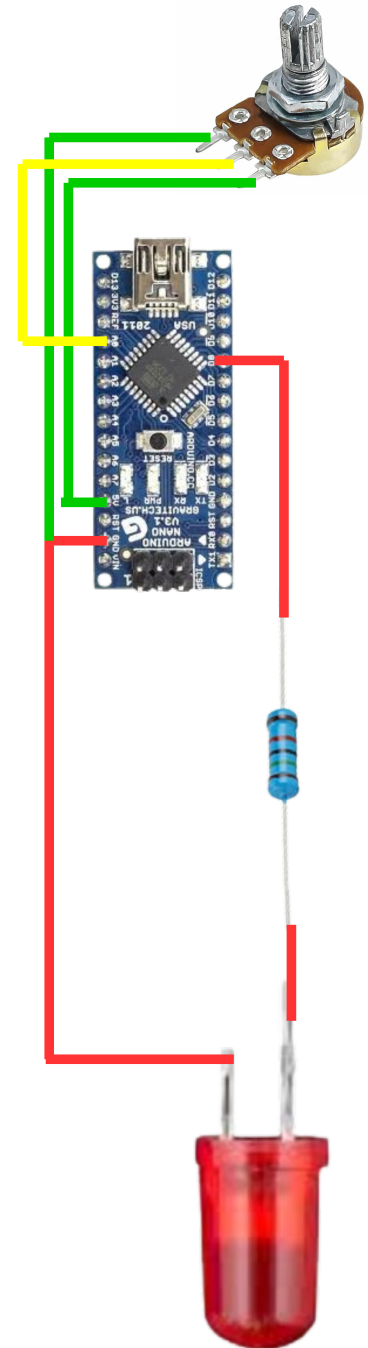
(Next slide)

Lab 4: Analog input

Steps (continued)

3. Run a wire from the “A0” pin of your development board to the middle row of the potentiometer.
4. Run a wire from the “GND” pin of your development board to either row of the potentiometer.
5. Run a wire from the “5V” pin on your development board to the remaining row on the potentiometer.
6. Copy and paste the code found at the following URL into the Arduino IDE's text editor window:
<https://raw.githubusercontent.com/dandandrea/intro-microcontrollers-lab-004/refs/heads/main/intro-microcontrollers-lab-004.ino>
7. Save the “Arduino Sketch” (program) and name it ***intro-microcontrollers-lab-004***
(File > Save)
8. Connect your development board to your laptop via the included USB cable.

(Next slide)



Lab 4: Analog input

Steps (continued)

9. Program your development board with the program displayed in Arduino IDE.

(Sketch > Upload)

10. Observe that as you turn the knob to one direction, the LED blinks increasingly faster, and that as you turn the knob in the other direction, the LED blinks increasingly slower.

11. Open the Arduino IDE's Serial Monitor.

(Tools > Serial Monitor)

12. Observe that as you turn the knob to one direction, an increasingly larger number is shown in the Serial Monitor, and that as you turn the knob in the other direction, an increasingly smaller number is shown. The smallest number shown should be zero (or very close to zero) and the largest number shown should be 1,023 (or very close to 1,023). Note that 1,023 is $2^{10} - 1$ or “2 to the 10th power minus 1”, with 10 representing how many bits of resolution the ADC has and minus 1 to account for the number starting at 0 instead of 1.

